Day 25: Transaction Tuple Record

1. Day Number

Day 25

2. Topic Name

Tuples

Today you will learn how to store related values together using a **tuple**.

---

3. Connection

Earlier, you stored transactions using individual strings, numbers, and lists.

Today, you will store **one transaction as a fixed record**.

For example, a transaction can contain:

- Transaction type: `"Deposit"`
- Amount: `500`

These two values can be stored together inside one tuple.

```
("Deposit", 500)
```

---

4. Important Topics

- Tuple
- Indexing
- Fixed data

---

5. Foundational Notes

What is a tuple?

A tuple is a collection that stores multiple values together.

A tuple normally uses parentheses:

```
("Deposit", 500)
```

This tuple contains two values:

Index	Value
0	`"Deposit"`
1	`500`

Tuple indexing

Tuple indexing starts from `0`.

```
transaction = ("Deposit", 500)

print(transaction[0])
print(transaction[1])
```

Here:

- `transaction[0]` gives the transaction type.
- `transaction[1]` gives the amount.

Tuples are fixed

Tuples are commonly used when a group of values represents one fixed record.

After creating a tuple, you should not try to change its individual values.

For example, this is not allowed:

```
transaction[0] = "Withdrawal"
```

Python will produce an error because tuples are **immutable**, meaning their values cannot be changed directly.

Tuple versus list

A list uses square brackets:

```
["Deposit", 500]
```

A tuple uses parentheses:

```
("Deposit", 500)
```

Use a tuple when the values form a fixed record and should not change accidentally.

---

6. Easy Example

Suppose you want to store a book record containing its title and price:

```
book = ("Python Basics", 20)

print(book[0])
print(book[1])
```

Output:

```
Python Basics
20
```

The same idea can be used for a transaction record.

---

7. Problem Statement

Create one transaction tuple containing:

- Transaction type
- Transaction amount

Example structure:

```
("Deposit", 500)
```

Print the transaction type and transaction amount separately.

Do not print the entire tuple as one value.

---

8. Concepts Used

Tuple creation

Store multiple related values inside parentheses.

```
(value1, value2)
```

String

The transaction type can be stored as a string.

Example:

```
"Deposit"
```

Number

The transaction amount can be stored as an integer or float.

Example:

```
500
```

Indexing

Access tuple values using their index positions:

```
record[0]
record[1]
```

Function

Place the transaction-display logic inside a function.

---

9. Thought Process

Think about the problem in these steps:

1. A transaction contains two related pieces of information.
2. The first value is the transaction type.

3. The second value is the transaction amount.
4. Store both values together in one tuple.
5. Pass the tuple to a function.
6. Access index `0` for the transaction type.
7. Access index `1` for the amount.
8. Print both values separately.

A transaction tuple can be visualized like this:

```
transaction
     |
     v
+-------------+--------+
| "Deposit"   | 500    |
+-------------+--------+
       0          1
```

---

10. Pseudocode

```
START

CREATE a function that accepts one transaction tuple

    GET transaction type from index 0
    GET transaction amount from index 1

    PRINT transaction type
    PRINT transaction amount

END function

CREATE one transaction tuple
CALL the function and pass the tuple

END
```

---

11. Suggested Solving Approach: Functional Approach

Create a function responsible for printing the transaction details.

Possible function responsibility:

```
display_transaction(transaction)
```

The function should:

1. Receive a transaction tuple.
2. Read the transaction type using index `0`.
3. Read the transaction amount using index `1`.
4. Print both values separately.

The main part of the program should:

1. Create the transaction tuple.
2. Call the function.

This keeps tuple creation and transaction display logically organized.

---

12. Easy Edge Cases

Edge Case 1: Wrong index idea

A tuple with two values has indexes `0` and `1`.

```
("Deposit", 500)
      0        1
```

Trying to access index `2` will produce an `IndexError` because there is no third value.

Incorrect idea:

```
transaction[2]
```

Edge Case 2: Amount is `0`

An amount of `0` is still a valid number and can be stored in the tuple.

Example:

```
("Deposit", 0)
```

Expected display:

```
Transaction type: Deposit
Amount: 0
```

Do not confuse `0` with a missing tuple value.

---

13. Expected Input

For this beginner version, no user input is required.

The transaction values can be directly stored in the program.

Example data:

```
Transaction type: Deposit
Amount: 500
```

Conceptual tuple:

```
("Deposit", 500)
```

---

14. Expected Output

```
Transaction type: Deposit
Amount: 500
```

For a zero-amount transaction:

```
Transaction type: Deposit
Amount: 0
```

---

15. Hint Only

Create a tuple with two values:

```
(transaction_type, amount)
```

Inside your function:

- Use index `0` to access the transaction type.
- Use index `1` to access the amount.
- Print each value with a clear label.

Remember:

```
First value  → index 0
Second value → index 1
```